Name: Pranjali Ingle Class: SOC-08 Batch: B

Subject: PL Roll No.: 43 Exp. No.: 2

Name of the Experiment:

Performed on:

Submitted on:

Marks	Teacher's Signature with date
10	

PL - Assignment 2

1 list of Python sequence data type and state one key Characteristic at each with respect to mutability or ordering.

→ data type are: String (Str): Immutable and ordered Collection of characters.

- list: Mutable & ordered Collection or elements
- Tuple: immutable and ordered Collection of elements
- Range: Immutable and ordered sequence of numbers.

2. Explain how indexing and slicing work in Python sequences, Illustrate how they differ when applied to strings & lists.

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐

Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐

Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

→ Indexing is used to access a single element of at a sequence using its position. Positive indexing starts from 1.

Slicing extracts a part of sequence using the syntax.

Difference in strings & lists:

- In strings, indexing / slicing returns characters or substrings
- In lists, indexing / slicing returns elements or sublists.

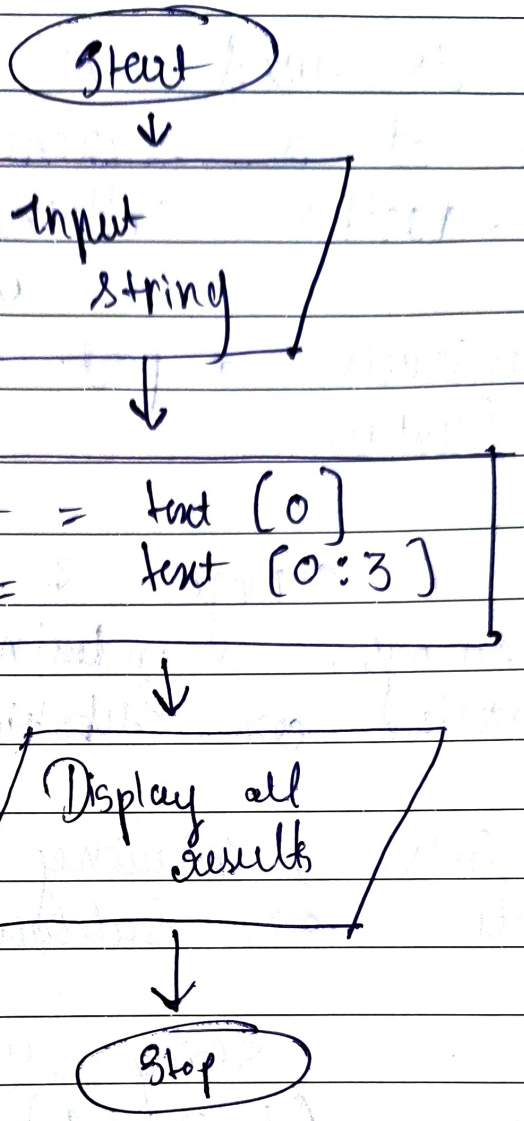
example: String :- "Python" [1:4] → "yth"
list : (1, 2, 3, 4) [1:3] → [2, 3]

3 Python program using indexing, slicing & built in functions.

→ Code:

```
text = "engineering"
first = Char = text (0)
Sub - text = text [0:5]
length = len (text)
upper - text = text, upper ()

Print ("first character:", first - Char)
Print ("Sliced text:" Sub - text)
Print ("length:" length)
Print ("upper case:" upper text)
```



4] Difference between equality (==) & Identity (is)

→ == (equality operator) checks whether two sequence refer to the same object in memory.

eg. a = [1, 2]
b = [1, 2] → a == b is True, but a is false.

5 Evaluation of lists, sets & dictionaries for student records.

→ • lists allow duplicates and are ordered, but searching is slower.

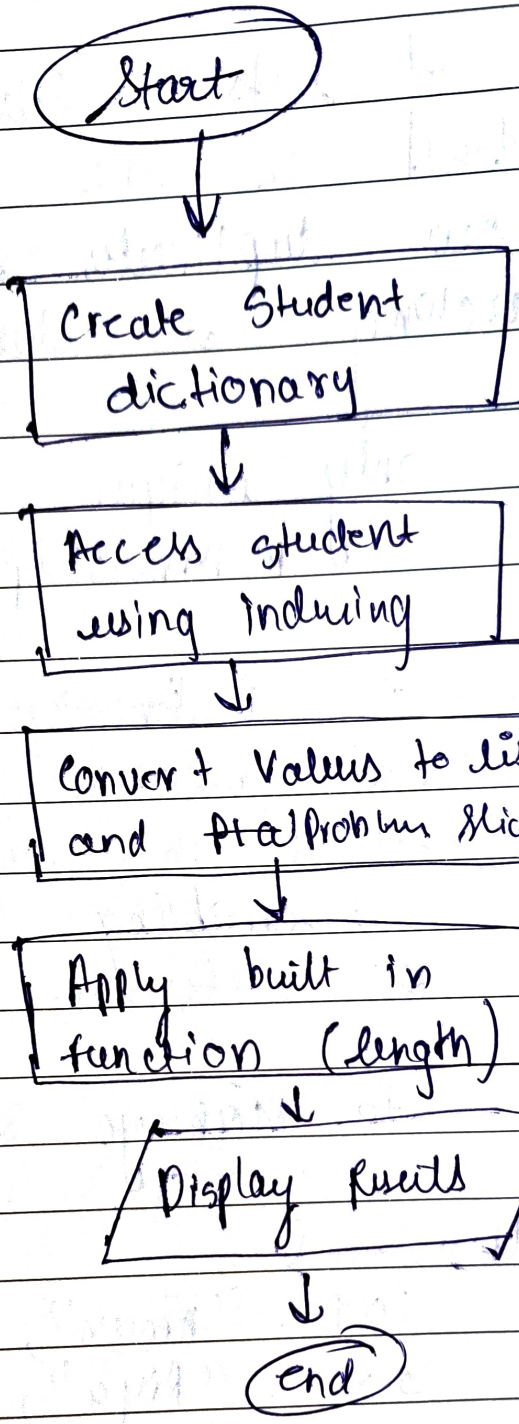
• sets store only unique values & are fast for membership testing.

Most efficient :- Dictionaries are best for student records because each student can be stored using unique ID as a key, enabling fast lookup, update & retrieval.

6 Program to Manage student data.

→

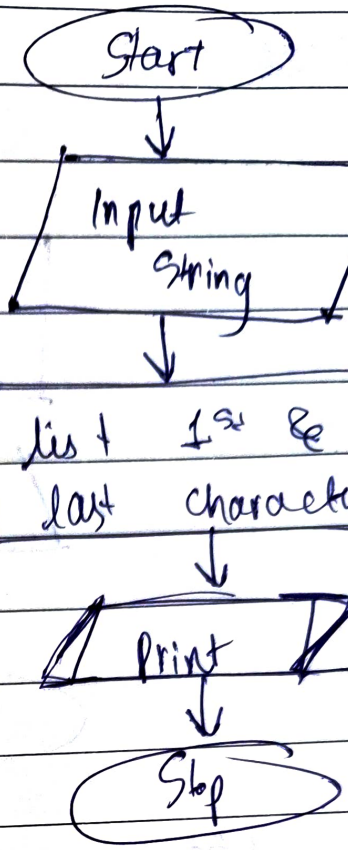
```
students = {
    101 : "Aman",
    102 : "Riya",
    103 : "Karan",
}
Print ("Student with ID 102 :" Student [01]
names = list (students. Values ())
Print (" first two students : ", names [0,2])
Print (" total students :", ten (students))
```



4 Print 1st & last Character of a String.

→ Algorithm : 1. Start
2. Input a String
3. Print 1st character using index 0
4. Print last character using index -1
5. Stop.

flowchart :-

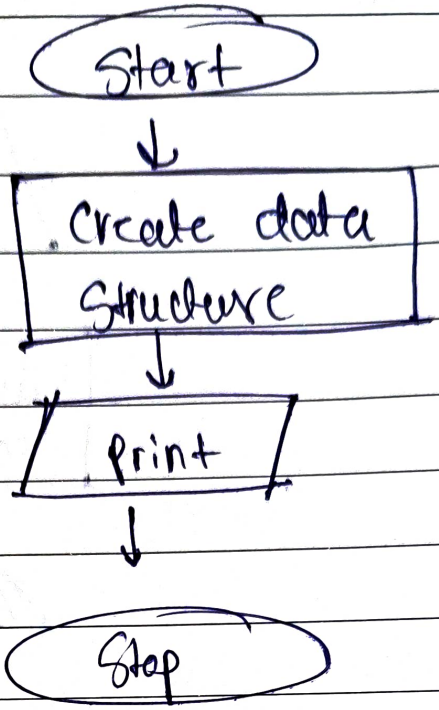


output :- enter a string : Python.
first character : P
last character : n.

8 Write a program to create list, tuple, set & dictionary.

→ Algorithm : 1. Start
2. Create list, tuple, set dictionary
3. Display them
4. stop.

Flowchart



Code:
```
lst = [1, 2, 3]
tup = (4, 5, 6)
st = {7, 8, 9}
dct = {"a": 1, "b": 2}
```

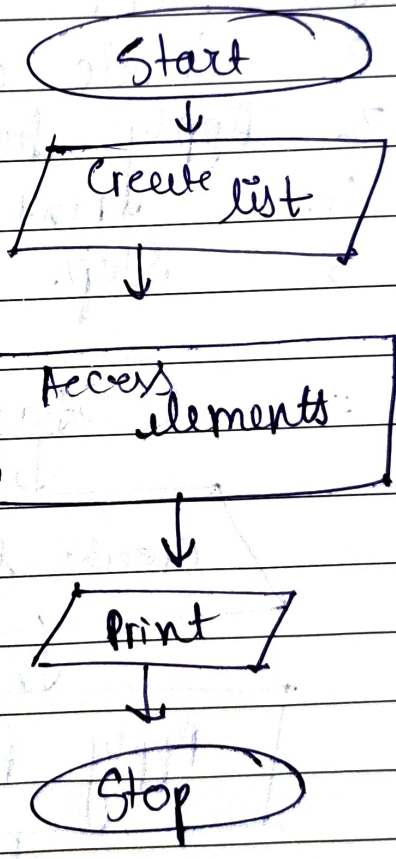
Output =
```
List :- [1, 2, 3]
Tuple :- (4, 5, 6)
Set :- {7, 8, 9}
Dictionary :- {'a': 1 'b': 2}
```

9 → Program to understand indexing

Algorithm:

1. Start
2. Create a list
3. Access elements using indexing
4. Print elements
5. Stop.

Flowchart :



Code:

```
numbers = [10, 20, 30, 40]
Print ("First element:", numbers[0])
Print ("Second element:", numbers[1])
Print ("Last element:", numbers[-1])
```

output

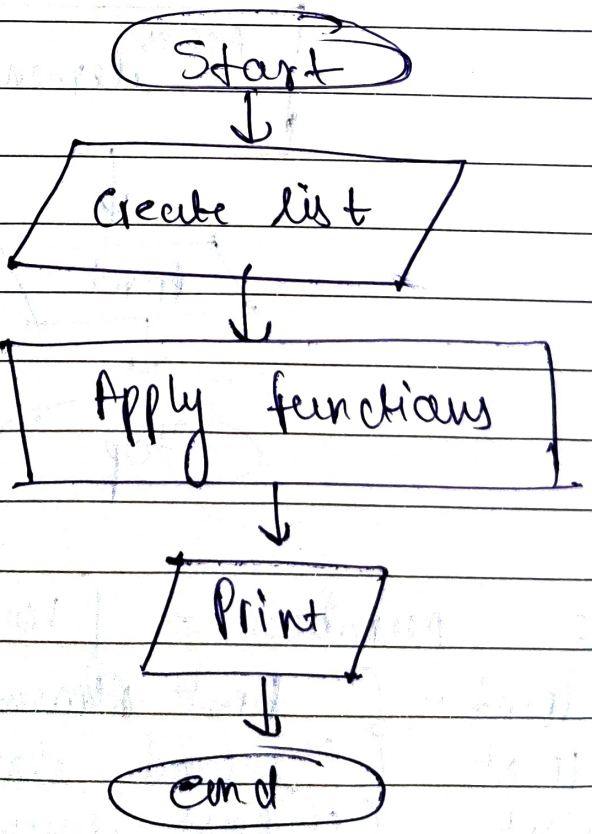
first element : 10
Second element : 20
Third element : 30

10] Program to understand built in function.

→ Algorithm.

1. Start
2. Create a list
3. Apply built in functions
4. Print results
5. Stop.

flowchart :-



code :- numbers = [5, 10, 15, 20]

```
Print ("length", len (numbers))
Print ("Maximum :", Max (numbers))
Print ("Minimum : ", Min (numbers))
Print ("Sum : ", Sum, (numbers))
```

output :- length : 4
Max : 20
Min : 5
Sum : 50.